

PROJECT EXPERIENCE REPORT

(After Action)

ToolShare

ENSE 477

Team Name : ToolShare

Project Reviewed : SSE Capstone

Group Participants:

Name:	Focus:
Jay Patel	Frontend Developer and Documentation
Isaac Kydd	Backend Developer and Database

Introduction

Our project is called “Toolshare”. It is a program designed to help universities manage their items, and make it easier for faculty to both make their items available and known to students while also making it easier to manage and keep track of them. We also want to make it easier for students to see what is available to them, streamline the borrowing process, and make it easier to understand what they do and don’t have access to. We did this by using the following tech stack: React & Vite for the frontend, MariaDB (MySQL) for the database, Express & NodeJS for the backend and AWS for deployment. Isaac Kydd is primarily the backend developer, and Jay Patel is primarily the frontend developer. We have created this project over the last few months. The following report will describe both the design process, how we got the idea in the first place, as well as what we did in order to achieve our goals.

Why we made it

Our project is called “Toolshare”. It started as a simple idea for a community item borrowing service. What we initially envisioned was a website and/or app that would allow people to put up equipment that they owned for people that lived in their neighborhood/community to borrow, maybe for a small fee. This would primarily be useful for things like lawnmowers, snowblowers, things that are really expensive, but you don’t use very often. So instead of spending 300\$ on a lawnmower, you could just pay somebody 10\$ or so to borrow their lawnmower for half an hour. The purpose was both to create a stronger sense of community coherence as well as to help people save money. However, over time, our ideas changed. We decided to see how the University of Regina managed their item borrowing and booking system, to get an idea of what something like this might look like. However, we quickly realised that the University did not have a unified booking system, and that every department used their own way of doing things. Some departments do everything on pen and paper, some use excel, and some use some 3rd party software to help them manage it. Often it was a combination of multiple methods.

The project quickly changed in scope to be focused on Universities. We wanted to help Universities (specifically the University of Regina) be able to have software dedicated to be able to help them lend out items more easily. We would build it with universities in mind. Things like separate faculties, grade level of students, etc., would be taken into account to ensure that all of the needs of a University would be met.

What we built

What we built could be described as a CRUD that is specifically optimized to meet the needs of universities. This means things like only faculty/approved users should be able to add items to the website (if university students could add whatever they wanted, things would go wrong very quickly). We have implemented rules based access and control systems, so that only certain types of users can do certain things. There are two primary roles, faculty and students. The Faculty can add items to the website, giving them a title, description, category, and an image. They can edit information about their items after they have been uploaded as well, and even delete them if they are no longer able to be used. They can also view the requests made for their items, and either approve or reject those requests. If a request is approved, the student is notified via email & notification on the webpage itself that their request was approved. Similarly, a denied request will give a notification, along with the reason why their request was rejected. When the student arrives to pick up their item at their requested time, the faculty member will go to their order and click on “Check Out”. They will then be prompted to add a “condition image”. What this means is an image of the item as it currently exists, which would include any existing damage. This is so that if an item is taken out in a damaged state, and then returned in the same state, the student cannot be blamed for damage that they did not cause. Instead of the student insisting “it was like that when I got it” and the faculty insisting that there was no record of it being damaged, they would have an image to directly compare it with so they could know for sure. Similarly, if the item was in good condition, then brought back broken, there would be a record of its prior state as well. When the item is returned at the agreed upon time, the faculty member clicks on “Return” to return the

item, then enters another condition image so that there is a record of the item's state both at the moment of it being checked out, as well as when it is returned.

From the student's perspective, after logging in, they will be able to browse the list of available items. They can also sort by searching for an item's name, sorting by categories, and even by an item's available times. For example, if they need an item from April 19th at noon to April 20th at 3pm, they can enter that into the item search filter, and all the items that already have bookings within that time frame will not show up. When they want to borrow an item, they click "Add to basket". This will add their item to their current basket, similar to a shopping cart on a website like amazon. The student can borrow multiple items at once. After they have selected all of their items, they click on the basket icon to go to the "checkout" page. There they will both verify that they understand the terms of service, that they are liable for damage they may cause to the items, etc. Then they will enter the times they wish to book the items. They can also choose to enter different times for different items, if they do not need them all at once. Then they must enter the reason for their request, which will be visible to the faculty member(s) viewing the request. If the faculty member decides that their reason for the items is not good enough, or it is an inappropriate use for them, they may reject the request. Item acceptance/rejection is also individually done, even if multiple items were requested at once, so it is possible to have some items approved and some rejected. Upon either approval or rejection, the student will be notified via email about the status of their request. From there they can pick it up and the requested time, then (hopefully) bring it back at the time they started as well.

Things like late fees and damaged items would be dealt with internally by the university. Currently there is no mechanism in the application to bill students. With

the condition image system showing a record of item damage, or the system showing an item was returned late. The University could simply bill the student for the cost to replace/repair the item, or any late fees associated with returning items late. While this is not a super exhaustive explanation of every feature our application offers, this is a fairly decent explanation of what we have built.

How we built it

We built this program using our specific tech stack because it was both geared towards what we envisioned making and also because it was technologies we were familiar with from our experiences in the past. MySQL itself or a MySQL derivative is the obvious choice for data storage structured in the way we envisioned it.

For the frontend, we chose Vite and React. React's component-based structure was a natural fit for the kind of interface ToolShare required: a page with many interactive, stateful elements like item filters, a basket, a multi-step checkout, and a notification system. Vite was chosen as the build tool because of its significantly faster development server performance compared to alternatives like Create React App, which made the day-to-day development experience much smoother.

We used NodeJS and Express because they were things we were familiar with, and knew that they would be adequate for constructing the backend as well as managing database interactions. They are used in order to get information submitted in the webpage to the database, and manage how that data is transferred. We also use it to verify the integrity of who is logged in, and things like the JSON Web Token (JWT) in order to authenticate and grant access to the proper users. While a complete breakdown of every route is impossible in this report, this is generally how we decided to build the backend of our project.

Development Process

Development started with the creation of several low-fidelity and high-fidelity prototypes. We wanted to ensure that what we envisioned for our project was a design that matched what users would need. One very essential “customer” that we consulted was the equipment manager of one of the Universities faculties. When asked about their booking process and after hearing about our project idea, they expressed interest in the project. We began running our design ideas as well as our low/high fidelity prototypes past them in order to get a better understanding of what an equipment manager would need. Eventually we started prototyping. First came the rough system architecture to determine how data would be dealt with. We also created a rough database schematic to decide how the stored data would be split up for both efficient retrieval, as well as it being intuitive to understand so that there was no confusion about where a piece of data would be located. We initially worked in two separate ways. One of us was working on a backend/database for the project, while the other worked on building a presentable frontend for the project. Initially this caused issues, because development was not synced through something like github. However, we eventually merged our ends of the project into a single project that we would work on throughout the remaining time. If we did this again, we would begin by using a unified codebase, like github, so that we were both operating off of the latest changes made by the other person. Initial prototyping was done on our local machines, downloading source code and synchronising databases in order to ensure that we were both working off of the same system.

The development of the project was largely divided into SCRUMs. We would make a two-week plan for development, then at the end of those two weeks, we would report

on how well we accomplished our goals. If we had adequately done everything we had planned and documented the process well, then we would move on to the next features that we wanted to implement. If we had not completed our goals as we had planned and/or had not properly documented what we had done, we would take note of the situation, and an extra effort would be made in the following 2 week period to both accomplish what we had missed (as well as complete appropriate documentation), and complete the next period's design goals as well.

Eventually we would deploy an early MVP (minimum viable product) on an AWS instance. This changed how we developed the project, as we began making changes with the understanding that they would need to work on an online production, not just on a localhost server. Any change that was made could be seen live immediately by both of us.

Challenges

One of the challenges we faced is that neither of us had ever deployed a project to AWS before. However, we had taken classes on how to configure a Linux instance and server webpages from it, as well as how to set up a database, which proved invaluable for this project. However, many of the AWS-specific things that we needed to do in order to get things to run were not explicitly taught, so much of it had to be figured out as we went. An example of this is that the port used to send mail is blocked by AWS, as they do not want their service to be used for unlimited spam. The solution was to use an email relay to send the emails through a gmail account, but this took us some time to figure out.

Testing

Manual testing was done throughout project development. When we would add a new feature or change something, we would make sure the new feature worked, then we would go through and make sure that any related features still worked as intended, and that there weren't unintended errors as a result of the new code. However, we did not do this systematically. Instead of using proper test-driven development and having test cases that would be used throughout the project lifecycle to do systematic regression testing, most of our testing was ad-hoc. A formal test suite was not created until well into the project's development. Unfortunately, this meant that we ran into issues when making changes, then realising some time later that something had broken, but not immediately being able to tell what the culprit was. We realise now that had we done proper test-driven development from the beginning, a lot of time would likely have been saved throughout the project.

In general though, every time a major feature was added, we would test the entire system manually as thoroughly as we could, although maybe not as efficiently as a proper testing suite, we still caught many errors and issues with the code before they became massive problems. For a project of this size, the way we did our early -development and mid-development testing was very helpful for the project. When a new UI element was added to the frontend, we would try to break it. What would happen if we used inspect element to change the value of an input to something unintended? What if we put in absurd numbers? To give an example, there is a feature to let faculty add items to the website. However, we eventually added the ability to add copies of the same item by entering a "quantity" value. If you left it as 1, only 1 item would be added, if you set it to 3, then 3 copies of that item would be added, etc. We tried putting in a value like "j" to see what would happen. Thankfully the query

that resulted from that gave a server error and nothing was entered into the database. Then we modified that field so that even with browser editing tools, a value like that could not be inserted into that field. But even if it was, the server would simply reject it.

Near the end of the development cycle, it became necessary to do a more exhaustive sort of testing on the system. This led to the creation of (at the time of writing) 100 test cases. These test cases consisted both of unit tests to test individual functions and SQL queries, and integration tests that combined many of the systems together. These tests revealed some faults in our code, which we have since corrected. However, we are aware that bug testing cannot prove that there are no errors in the code, only that we have eliminated particular possible issues. Testing should be ongoing, and the testing suite that has been created should be useful in future development, both for regression testing to ensure that previous features still work as intended when the code is modified for new features, as well as for testing out new features by adding more test cases as they become necessary.

What went well

The stakeholder consultation with the faculty of film was one of the most valuable parts of the process. Having a real user look at our prototypes and give feedback meant our design was grounded in actual needs rather than assumptions. Several features that make ToolShare genuinely useful – the condition image workflow, per-item approval, and the reason field on requests were either introduced or shaped directly by that feedback.

The basket and checkout flow also came together well. Designing the borrowing process to feel similar to a familiar e-commerce experience made the interface

intuitive. Users who tested the system were able to complete the full borrowing flow without needing any guidance.

Moving to a shared GitHub repository, even though it came later than it should have, transformed how we worked together. From that point on there were no more integration surprises, and both of us always had a clear picture of the current state of the codebase.

Successfully deploying to AWS was also a meaningful milestone. Getting a real production environment running with a live database, HTTP, and working email notifications gave us both experience with full-stack deployment that goes well beyond what most course projects provide.

Finally, the overall architecture of the system held up well throughout development. The three-tier separation of frontend, backend, and database proved clean and maintainable. Adding new features rarely required changes across all three layers at once, which made development faster and reduced the risk of unintended side effects.

What we would change

While the project was largely a success, there are a lot of things we would do differently in the future. First of all, more attention would be given to the design of the project before starting to write code. Some time was wasted near the beginning of the project's lifecycle writing code for prototypes much earlier than they were needed. While a lot of this code and part of the database structure were rolled over into the project's code once it was formally started, it would likely have been a better idea for finalize the design before beginning, as much of it went unused due to changing ideas for how we wanted to structure the project as a whole. While early prototyping can

give insights that theoretical design cannot, more focus should have been given to the design, especially early on.

Another thing we would change is our testing methodology. A formalized testing suite from the beginning would have saved a ton of time when performing regression testing after adding new features. While momentarily time-consuming to do, it would have saved a large amount of time in the long run. We did not have this perspective at the beginning, as there was a rush to “get the next feature done” as soon as possible. However, hindsight gives us the ability to recognize that this was a mistake, and test-driven development would have likely been a massive advantage for the development process.

The program as it exists today

The project as it exists today reflects our original design goals quite well. We built a robust system that allows faculty to easily make their items available to students, while allowing students to easily request them.

The application is deployed and running on AWS. Both the faculty and student interfaces are complete and functional. Faculty can add, edit, and remove items, review and action requests, and manage the full check-out and return workflow including condition images. Students can browse and filter the item catalogue, add items to a basket, submit requests with specified time windows, and receive email and in-app notifications on the outcome of their requests.

The formal test suite of 100 test cases covers the core functionality of the system and provides a foundation for ongoing regression testing as the project continues to develop. Known limitations include the absence of a built-in billing mechanism for

late fees or item damage, which is currently handled administratively by the university using the records ToolShare provides. The system also does not have a dedicated mobile application, though the web application works on mobile browsers.

Future development directions could include a dedicated mobile app, integration with university student information systems for automated user provisioning, more advanced scheduling views, and expanded reporting tools to help departments track item utilization over time.

Conclusion

ToolShare started as a simple neighbourhood borrowing idea and grew into a university equipment management platform. That shift happened because we looked closely at a real problem, which is that the University of Regina has no unified system for lending equipment to students, and we decided we were in a position to build one.

The project gave us experience that goes well beyond a normal course assignment. We had real stakeholder consultation, full-stack cloud deployment, a role-based access system built for a real institutional environment, and we learned hard lessons that come from discovering the limits of ad-hoc testing and working without version control from the start.

The most important things we are taking away from this project are not the specific features we built, but the process lessons we learned. Version control from day one. Test-driven development from day one. Talk to real users before finalizing your design. These are principles we will carry into every future project.

ToolShare is a system that a real university department expressed genuine interest in using. That, more than any technical achievement, is the most meaningful measure of the project's success.